

Informe Grupal: Prueba(Parte II) - Desarrollo grupal de estrategias

Grupo – INF8090 Computación Paralela y Distribuida

Mayo 2026

Datos del Grupo

Integrante	Correo / Identificación
Elliot Acevedo Zuñiga	eacevedoz@utem.cl
Benjamin Aballay Palma 2	baballay@utem.cl
Felipe Martinez Gonzalez 3	fmartinezgo@utem.cl

Curso: INF8090 – Computación Paralela y Distribuida

Evaluación: Prueba 1 – Parte II Grupal

Fecha: Mayo 2026

1. Ejercicio 1: Normalización Z-Score con OpenMP en C++

1.1. Diagnóstico y Selección de Estrategia

Durante la evaluación individual (Parte I), no se dejó registro de una estrategia preliminar escrita para este ejercicio. Por lo tanto, al iniciar la Parte II, el grupo partió desde cero para evaluar cómo abordar el cálculo de la media, varianza y normalización Z-Score sobre 6.4 GB de datos.

Al discutir las alternativas como grupo, nos planteamos inicialmente un enfoque ingenuo: paralelizar el cálculo directamente usando `#pragma omp critical` o `atomic` para ir sumando la media y varianza global a medida que se leía cada fila.

Sin embargo, descartamos esa opción y llegamos a la conclusión de que la mejor forma de hacerlo era mediante reducciones y variables locales (Estrategia de Memoria Compartida Local) por las siguientes razones técnicas:

- **Control de Dependencias y Sincronización:** Hacer un bloqueo constante (`critical`) por cada celda válida obligaría a los hilos a esperar turno millones de veces, serializando el código y destruyendo el rendimiento. En su lugar, decidimos usar la directiva `reduction(+:...)` de OpenMP para las sumas globales (media y varianza). OpenMP maneja los acumuladores privados por hilo en bajo nivel y los suma al final de forma ultra eficiente.
- **Manejo de Localidad y Memoria:** Como este es un problema fuertemente limitado por el ancho de banda de la memoria (*memory-bound*), estructuramos el código repartiendo las filas con un *scheduling* estático (`#pragma omp parallel for`). Esto permite que los núcleos de la CPU trabajen sin el problema de *false sharing*, aprovechando la caché local antes de sincronizar resultados.

1.2. Diagnóstico del Problema

Se requiere normalizar una matriz de punto flotante de dimensión $50\,000\,000 \times 16$ (~ 6.4 GB en precisión doble) que contiene valores faltantes (NaN) en aproximadamente el 5 % de sus celdas. La normalización Z-Score se define como:

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j} \quad \mu_j = \frac{1}{N_j} \sum_{i \notin \text{NaN}} x_{ij} \quad \sigma_j = \sqrt{\frac{1}{N_j} \sum_{i \notin \text{NaN}} (x_{ij} - \mu_j)^2}$$

donde μ_j y σ_j se calculan ignorando los NaN, y los NaN se preservan en la salida. Además, se cuentan los valores atípicos (*outliers*) definidos como $|z_{ij}| > 3$.

El problema presenta tres desafíos fundamentales:

1. **Volumen de datos:** 6.4 GB excede la caché de cualquier CPU moderna, por lo que el cuello de botella es el ancho de banda de la memoria principal (DRAM), no la capacidad de cómputo.
2. **Manejo de NaN:** Cada celda debe verificarse con `isnan()` antes de contribuir a los estadísticos o a la normalización.
3. **Tres fases de recorrido:** A diferencia de la normalización min-max (que requiere una pasada de min/max y otra de normalización), el Z-Score exige tres pasadas completas: media, varianza y normalización, incrementando el tráfico de memoria en $\sim 33\%$.

1.3. Datos

Se generó un dataset sintético de $50\,000\,000 \times 16$ a partir de coordenadas reales de viajes de Uber (archivos CSV públicos con > 2 millones de registros). Cada fila representa un viaje con 16 atributos numéricos: coordenadas de recogida y destino, distancia, tarifa, propina, peajes, velocidad, tiempo en tráfico, duración, multiplicador *surge*, cantidad de pasajeros y calificación del conductor. Se inyectó 5 % de NaN aleatorio. El archivo se guardó en formato binario plano (`double[]`) para lectura directa desde C++.

1.4. Estrategia de Solución

Se diseñó un pipeline de tres fases, cada una implementada con private arrays por hilo para evitar *false sharing* y una sección `critical` para combinar al final:

1. **Fase 1 (Media y conteo válido):** Recorrido de lectura con arreglos privados `local_sums[NCOLS]` y `local_vals[NCOLS]`. Cada hilo acumula sumas y conteos en sus arreglos locales. Al finalizar, se combinan en una sección `critical`. Luego $\mu_j = \text{sum}_j / \text{count}_j$.
2. **Fase 2 (Varianza):** Segundo recorrido de lectura similar, acumulando $\sum (x_{ij} - \mu_j)^2$ en arreglos privados `local_sum_sq[NCOLS]` y combinando con `critical`. Luego $\sigma_j = \sqrt{\text{sum_sq}_j / \text{count}_j}$.
3. **Fase 3 (Normalización Z-Score y detección de atípicos):** Tercer recorrido con `#pragma omp parallel for reduction(+:outliers)`. Cada celda válida se transforma a $z = (x - \mu_j) / \sigma_j$; se cuenta como atípico si $|z| > 3$. Los NaN se preservan.

1.4.1. Justificación Técnica de OpenMP

Se seleccionó OpenMP sobre MPI o hilos POSIX por las siguientes razones:

- **Memoria compartida:** Toda la matriz cabe en la RAM del sistema (15 GB disponibles), por lo que no se requiere distribución entre nodos.
- **Directivas de alto nivel:** `parallel for` y `critical` permiten paralelizar con mínima intervención manual sobre sincronización.
- **Granularidad gruesa:** El trabajo por fila es homogéneo (cada fila procesa 16 doubles), por lo que `schedule(static)` por defecto balancea bien la carga.
- **Escalabilidad esperada:** Al ser un problema *memory-bound*, se anticipa que el speedup esté limitado por el ancho de banda de memoria, no por el número de núcleos.

1.4.2. Uso de reduction y arreglos privados

Para evitar *false sharing* y condiciones de carrera, las Fases 1 y 2 utilizan arreglos privados por hilo (`local_sums`, `local_vals`, `local_sum_sq`) en una región `parallel`, combinando al final con una sección `critical` (solo 16 sumas por hilo, overhead despreciable). La Fase 3 usa `reduction(+:outliers)` que crea copias privadas del contador y las combina al finalizar el bucle.

1.5. Benchmark y Resultados

1.5.1. Registro del Entorno de Ejecución

- **Sistema operativo:** Linux 7.0.10-1-cachyos (Arch Linux)
- **CPU:** AMD Ryzen 7 5700G, 8 núcleos físicos, 16 hilos lógicos
- **RAM:** 15 GB total, ~11 GB disponibles
- **Compilador:** GCC 16.1.1 20260430
- **Soporte OpenMP:** OpenMP 5.1 (GCC libgomp)
- **Banderas de compilación:** `g++ -O3 -fopenmp -march=native`
- **Tamaño de datos:** $50\,000\,000 \times 16 = 6,4$ GB
- **Repeticiones:** Una corrida por configuración
- **Métrica principal:** Tiempo total = Fase 1 + Fase 2 + Fase 3

1.5.2. Tabla de Tiempos

Hilos	Fase 1 (s)	Fase 2 (s)	Fase 3 (s)	Total (s)	Speedup S_p
1	0.8340	0.7800	1.0272	2.6412	1.000
2	0.3126	0.4102	0.7544	1.4773	1.788
4	0.2059	0.2282	0.8009	1.2350	2.139
8	0.2469	0.2350	0.8331	1.3150	2.008

Cuadro 1: Tiempos de ejecución medidos y speedup respecto a 1 hilo.

1.5.3. Cálculo de Speedup y Eficiencia

$$S_p = \frac{T_1}{T_p} \quad E_p = \frac{S_p}{p}$$

p	T_p (s)	S_p	E_p (%)
1	2.6412	1.000	100.0
2	1.4773	1.788	89.4
4	1.2350	2.139	53.5
8	1.3150	2.008	25.1

Cuadro 2: Speedup y eficiencia para cada configuración de hilos.

1.5.4. Análisis de Resultados

El speedup máximo alcanzado es de $2,14\times$ con 4 hilos. Con 8 hilos el rendimiento decrece ligeramente ($2,01\times$). Este comportamiento es consistente con un problema limitado por ancho de banda de memoria:

- Las fases 1 y 2 son de solo lectura y escalan mejor: $S_4 \approx 3,6\times$ y $S_8 \approx 3,3\times$ en la fase 1, pues los 6.4 GB se leen desde DRAM sin contención de escritura.
- La fase 3 (normalización) lee y escribe 6.4 GB, saturando el bus de memoria. Con 1 hilo la velocidad efectiva es $6,4/1,0272 \approx 6,2$ GB/s; con 2 hilos sube a ~ 8.5 GB/s, y con 4 u 8 hilos se estabiliza en ~ 8 GB/s, indicando saturación del ancho de banda DDR4.
- El speedup total es menor que el del min-max original debido a las tres pasadas completas sobre los datos.
- Se detectaron 4734853 valores atípicos ($|Z| > 3$), que representan el 0.62% de los datos válidos.

1.5.5. Por qué critical dentro del bucle principal sería una mala decisión

Si se usara `#pragma omp critical` dentro del bucle de normalización para proteger la escritura de cada celda, se introduciría una serialización completa: todos los hilos competirían por la misma sección crítica, anulando cualquier beneficio del paralelismo. El overhead de entrar y salir de la sección crítica 800 millones de veces haría la ejecución mucho más lenta que la versión secuencial.

En nuestra implementación, la sección `critical` se usa solo una vez por hilo por fase para combinar 16 o 32 valores, lo que representa un overhead despreciable.

1.5.6. Error Numérico y Orden de Reducción en Punto Flotante

La suma de punto flotante no es asociativa debido a la precisión finita de IEEE 754. En las fases 1 y 2, los acumuladores privados se suman en orden indeterminado dentro de la sección `critical`, lo que puede producir diferencias del orden de 10^{-6} en μ_j y σ_j entre ejecuciones. Este error es despreciable para el análisis de datos y no afecta la clasificación de atípicos.

1.6. Conclusiones del Ejercicio 1

1. El problema es *memory-bound*. El speedup está limitado por el ancho de banda de memoria DDR4, no por la capacidad de cómputo.
2. El Z-Score requiere 3 pasadas completas sobre los datos, incrementando el tráfico de memoria en $\sim 33\%$ respecto a min-max.
3. OpenMP permite paralelizar eficientemente usando arreglos privados por hilo y secciones `critical` para combinar.
4. La estrategia fallaría en sistemas con menos RAM que el tamaño del dataset, requiriendo procesamiento por bloques con lectura a disco.

2. Ejercicio 2: Pipeline Concurrente de Procesamiento de Logs en Python

2.1. Diagnóstico y Selección de Estrategia

Durante la evaluación individual (Parte I), cada integrante propuso una estrategia distinta para este problema. Al comparar en grupo, las diferencias más relevantes giraron en torno a si usar `threading`, `multiprocessing` o un enfoque híbrido. La discusión nos llevó a descartar `threading` como solución principal y a elegir `ProcessPoolExecutor` con **particionamiento por chunks** por las siguientes razones:

- **El GIL bloquea las etapas más costosas:** las etapas de parseo, limpieza y agregación son CPU-bound y ejecutan Python puro. El GIL impide que dos hilos ejecuten bytecode Python simultáneamente, por lo que `threading` no introduce paralelismo real en estas fases.
- **La lectura puede ser secuencial:** la descompresión con `gzip` libera el GIL parcialmente (la rutina de inflado está en C), por lo que `threading` ayudaría en la lectura, pero ese beneficio es menor comparado con el costo del parseo.
- **El merge es seguro sin locks:** al usar el patrón *map-reduce* (cada worker retorna un diccionario parcial y el proceso principal los combina), se evitan condiciones de carrera sin ningún mecanismo de sincronización explícito. Esto es equivalente a `reduction` en OpenMP.

Descartamos `asyncio` porque los logs están en disco local, no en red.

2.2. Diagnóstico del Problema

Se requiere procesar 2 GB de logs comprimidos en formato JSONL con los campos: `user_id`, `timestamp`, `endpoint`, `latency_ms` y `status_code`. El objetivo es agregar métricas por ventanas temporales de 10 minutos.

Etapas	Tipo de carga	Razón
Lectura y descompresión	I/O-bound	Acceso a disco; <code>gzip</code> usa C interno
Parseo JSON	CPU-bound	<code>json.loads()</code> por línea, Python puro
Limpieza y validación	CPU-bound	Verificaciones de campos, conversiones de tipo
Agregación por ventanas	CPU-bound	Agrupaciones, sumas, conteos en diccionarios
Escritura de resultados	I/O-bound	Salida pequeña, no es cuello de botella

Cuadro 3: Tipo de carga por etapa del pipeline.

2.3. Dataset Sintético

Se generó un dataset sintético de aproximadamente 2 GB comprimidos (~ 50 millones de líneas) usando NumPy:

Listing 1: Generación de dataset sintético.

```

1 import gzip, json, numpy as np
2 from datetime import datetime, timedelta
3
4 def generate_logs(n=50_000_000, filename="logs.jsonl.gz"):
5     users = [f"user_{i}" for i in range(1000)]
6     endpoints = ["/api/search", "/api/checkout", "/api/login", "/api/data"]
7     codes = [200, 200, 200, 404, 500]
8     base = datetime(2024, 1, 1)
9     batch = 100_000
10    with gzip.open(filename, "wt") as f:
11        for start in range(0, n, batch):
12            size = min(batch, n - start)
13            u = np.random.randint(0, 1000, size)
14            e = np.random.randint(0, 4, size)
15            c = np.random.choice(codes, size)
16            lat = np.round(np.random.normal(120, 40, size), 2)
17            ts_offsets = np.arange(start, start + size) * 0.1
18            for i in range(size):
19                ts = (base + timedelta(
20                    seconds=float(ts_offsets[i]))).isoformat()
21                f.write(json.dumps({
22                    "user_id": users[u[i]],
23                    "timestamp": ts,
24                    "endpoint": endpoints[e[i]],
25                    "latency_ms": float(lat[i]),
26                    "status_code": int(c[i])
27                }) + "\n")

```

Nota sobre escala: el benchmark se ejecutó sobre 5 millones de líneas (~200 MB comprimidos) y los resultados se extrapolan linealmente a 50 millones, lo cual es válido porque el trabajo por línea es homogéneo.

2.4. Tabla Comparativa de Estrategias

Criterio			Secuencial	threading / ThreadPool	multiprocessing / ProcessPool	Híbrido (asyncio+MP)
Paralelismo	CPU	No	No	No (GIL)	Sí	Sí
Paralelismo	I/O	No	Sí	Sí	Sí	Sí
Overhead		Mínimo	Bajo	Alto	(fork+pickle)	Medio
Complejidad código		Baja	Baja	Media	Alta	
Útil parseo	CPU-bound	No	No	Sí	Sí	Sí
Útil lectura	I/O-bound	No	Sí	Sí	Sí	Sí
Recomendado		Baseline	No	Sí	Solo logs en red	

Cuadro 4: Comparación de estrategias de concurrencia.

2.5. Impacto del GIL

- **Lectura y descompresión:** zlib libera el GIL durante el inflado. **threading** sí paralela esta etapa parcialmente.
- **Parseo JSON:** `json.loads()` está en C pero crear objetos Python requiere el GIL. Beneficio marginal con **threading**.
- **Limpieza y agregación:** Python puro. El GIL bloquea completamente. Solo **multiprocessing** es efectivo.

2.6. Estrategia Seleccionada

2.6.1. Diseño de Particionamiento

Se particiona el archivo en chunks de 50.000 líneas. Cada chunk es una lista de strings enviada a un worker. El worker retorna un diccionario parcial con clave (`endpoint`, `ventana_10min`) y valor `{count, total_latency, errors}`. El proceso principal combina los diccionarios con una reducción secuencial sin locks.

2.6.2. Pseudocódigo

Listing 2: Pipeline con ProcessPoolExecutor.

```
1 from concurrent.futures import ProcessPoolExecutor
2 from collections import defaultdict
3 import gzip, json
4
5 def read_chunks(filepath, chunk_size=50_000):
6     with gzip.open(filepath, "rt") as f:
7         chunk = []
8         for line in f:
9             chunk.append(line)
10            if len(chunk) >= chunk_size:
11                yield chunk
12                chunk = []
13        if chunk:
14            yield chunk
15
16 def process_chunk(lines):
17     partial = defaultdict(lambda: {"count": 0,
18                                     "total_latency": 0.0,
19                                     "errors": 0})
20
21     for line in lines:
22         try:
23             r = json.loads(line)
24             if r["latency_ms"] < 0:
25                 continue
26             key = (r["endpoint"], r["timestamp"][:15] + "0:00")
27             partial[key]["count"] += 1
28             partial[key]["total_latency"] += r["latency_ms"]
29             if r["status_code"] >= 400:
30                 partial[key]["errors"] += 1
31         except (json.JSONDecodeError, KeyError):
32             continue
33     return dict(partial)
34
35 def merge_results(partials):
36     final = defaultdict(lambda: {"count": 0,
37                                   "total_latency": 0.0,
38                                   "errors": 0})
39
40     for partial in partials:
41         for key, vals in partial.items():
42             final[key]["count"] += vals["count"]
43             final[key]["total_latency"] += vals["total_latency"]
44             final[key]["errors"] += vals["errors"]
45     return final
46
47 def run_pipeline(filepath, n_workers=4):
48     import time
49     t0 = time.perf_counter()
50     chunks = list(read_chunks(filepath))
51     with ProcessPoolExecutor(max_workers=n_workers) as pool:
52         partials = list(pool.map(process_chunk, chunks))
53     final = merge_results(partials)
```

```
return final, time.perf_counter() - t0
```

El merge no requiere locks porque todos los workers terminan antes de que `merge_results` comience (barrera implícita al salir del bloque `with`). Es el equivalente directo de `reduction(+:...)` en OpenMP.

2.7. Identificación de Overhead

Fuente	Dónde ocurre	Magnitud	Mitigación
Serialización pickle	Envío de chunks y recepción de dicts	Alta si chunks < 10K líneas	Chunks de 50K (≈ 5 MB)
Fork de procesos	Creación del pool	Fijo ~ 100 –300 ms	Reutilizar el pool
Copia IPC	Worker $\rightarrow$ proceso principal	Proporcional al dict parcial	Devolver solo agregaciones
Merge secuencial	Proceso principal	< 1 % del tiempo total	No paralelizar

Cuadro 5: Fuentes de overhead y mitigaciones.

2.8. Benchmark y Resultados

2.8.1. Registro del Entorno de Ejecución

- **Sistema operativo:** Windows 11
- **CPU:** AMD Ryzen 5 5500, 6 núcleos físicos, 12 hilos lógicos
- **RAM:** 32 GB
- **Versión Python:** 3.12.3
- **Tamaño de datos medido:** 5.000.000 líneas (≈ 200 MB comprimidos)
- **Tamaño de datos objetivo:** 50.000.000 líneas (≈ 2 GB comprimidos)
- **Repeticiones:** 5 por configuración; 2 warm-up descartadas
- **Chunk size:** 50.000 líneas
- **Métrica:** tiempo wall-clock total (lectura + procesamiento + merge)

2.8.2. Tabla de Tiempos Medidos (5M líneas)

Estrategia	Workers	Tiempo (s)	Speedup S_p	Eficiencia E_p (%)
Secuencial	1	15.57	1.00	100.0
threading	4	15.18	1.03	25.7
multiprocessing	2	8.31	1.87	93.7
multiprocessing	4	4.81	3.24	80.9
multiprocessing	8	3.44	4.53	56.7

Cuadro 6: Tiempos medidos sobre 5M líneas.

$$S_p = \frac{T_1}{T_p} \quad E_p = \frac{S_p}{p}$$

Con 4 workers: $S_4 = 15,57/4,81 = 3,24$, $E_4 = 80,9\%$. Con 8 workers: $S_8 = 15,57/3,44 = 4,53$, $E_8 = 56,7\%$.

2.8.3. Análisis de Resultados

- **threading no escala:** speedup de $1,03\times$ confirma que el GIL anula el paralelismo CPU-bound.
- **multiprocessing escala bien:** eficiencia de 93.7 % con 2 workers y 80.9 % con 4 workers, mostrando overhead moderado de pickle.
- **Saturación a 8 workers:** con 8 workers se supera el número de núcleos físicos (6), por lo que los workers adicionales operan sobre hilos lógicos via hyperthreading, explicando la menor eficiencia (56.7 %).
- **Extrapolación lineal válida:** el número de grupos de agregación es finito (≈ 35.000 grupos), no crece superlinealmente.

2.9. Riesgos y Limitaciones

Riesgo	Descripción	Condición de fallo
Cuello de botella en lectura	Lectura secuencial limita el throughput	Disco lento (HDD) o archivo en red
OOM en merge	Dict final crece si hay muchas claves únicas	Ventanas < 1 min o millones de endpoints
Overhead domina	Pickle supera el beneficio con chunks pequeños	chunk_size $< 10K$ con muchos workers
asyncio ineficaz	No ayuda con I/O de disco local	Solo si logs vienen de S3 o HTTP

Cuadro 7: Riesgos identificados y condiciones de fallo.

2.10. Conclusiones del Ejercicio 2

1. No existe una herramienta universal: **multiprocessing** es la única opción efectiva en Python para pipelines CPU-bound con logs en disco local.
2. El GIL hace que **threading** sea ineficaz en las etapas dominantes; el speedup real es $\approx 1,0$.
3. El patrón map-reduce elimina la necesidad de locks por diseño, equivalente a **reduction** en OpenMP.
4. El speedup máximo real es $\approx 4\times$ antes de saturar la lectura. Dividir el dataset en múltiples archivos permitiría paralelizar también la lectura.

3. Ejercicio 3: Cálculo de Similitud por Bloques para Embeddings

3.1. Diagnóstico y Selección de Estrategia

El problema consiste en calcular similitudes entre 20 000 embeddings de dimensión 128, normalizados a norma unitaria, y obtener para cada vector sus 10 vecinos más similares. La similitud utilizada corresponde a similitud coseno. Como todos los vectores están normalizados a norma 1, la similitud coseno se reduce al producto punto:

$$\text{sim}(x_i, x_j) = \frac{x_i \cdot x_j}{\|x_i\|_2 \|x_j\|_2} = x_i \cdot x_j$$

La estrategia ingenua consiste en construir la matriz completa de similitud $S = XX^T$, donde $X \in R^{20000 \times 128}$ y $S \in R^{20000 \times 20000}$. Sin embargo, esta estrategia almacena 400 000 000 valores aunque el objetivo final solo requiere 10 valores por fila. Por lo tanto, se descarta la materialización completa de la matriz y se selecciona una estrategia exacta basada en procesamiento por bloques y actualización incremental del top- k .

La solución implementada utiliza Python con NumPy y **multiprocessing**. No se usa g++ ni OpenMP en esta parte, porque el ejercicio exige una estrategia de bloques, memoria, top- k y métricas de rendimiento, pero no obliga a implementar OpenMP específicamente para este tercer ejercicio. Para evitar la limitación

del GIL de Python, se utilizan procesos mediante `ProcessPoolExecutor`. La matriz de embeddings se comparte entre procesos con `shared_memory`, evitando copiar el dataset completo a cada worker.

3.2. Diagnóstico del Problema

Sea $N = 20\,000$ el número de vectores y $D = 128$ la dimensión de cada embedding. Si se comparan todos los vectores contra todos los vectores, el número de comparaciones es:

$$N(N - 1) = 20\,000 \times 19\,999 = 399\,980\,000$$

Si se considera también la diagonal, la matriz completa tiene:

$$N^2 = 20\,000^2 = 400\,000\,000$$

entradas. Cada comparación requiere aproximadamente D multiplicaciones y sumas, por lo que el costo computacional es:

$$O(N^2 D)$$

Para este caso:

$$400\,000\,000 \times 128 = 51\,200\,000\,000$$

operaciones elementales de multiplicación-acumulación aproximadas. Por lo tanto, el problema no se resuelve simplemente paralelizando un doble bucle. La dificultad real está en controlar memoria, localidad de acceso, top- k parcial, reducción de candidatos y costo cuadrático.

3.3. Dataset Utilizado

Se generó un dataset sintético compuesto por 20 000 vectores de dimensión 128. Los valores se generaron con distribución normal estándar y posteriormente cada fila fue normalizada por su norma euclidiana:

$$x_i \leftarrow \frac{x_i}{\|x_i\|_2}$$

Esto garantiza que:

$$\|x_i\|_2 = 1$$

La generación sintética es adecuada para este ejercicio porque permite controlar exactamente el tamaño del problema, cumplir con el formato exigido y reproducir los resultados usando una semilla fija.

Listing 3: Generación de embeddings sintéticos normalizados.

```

1 def generar_embeddings(N, D, seed=42):
2     rng = np.random.default_rng(seed)
3     X = rng.normal(0, 1, size=(N, D)).astype(np.float32)
4
5     normas = np.linalg.norm(X, axis=1, keepdims=True)
6     normas[normas == 0] = 1.0
7
8     X = X / normas
9     return X.astype(np.float32)
```

3.4. Estimación de Memoria

3.4.1. Matriz completa de similitud

La matriz completa de similitud tendría 400 000 000 elementos. En precisión `float32`, cada elemento ocupa 4 bytes:

$$400\,000\,000 \times 4 = 1\,600\,000\,000 \text{ bytes}$$

Esto equivale aproximadamente a:

$$1,49 \text{ GiB} \approx 1,6 \text{ GB}$$

En `float64`, cada elemento ocupa 8 bytes:

$$400\,000\,000 \times 8 = 3\,200\,000\,000 \text{ bytes}$$

Esto equivale aproximadamente a:

$$2,98 \text{ GiB} \approx 3,2 \text{ GB}$$

Estas cifras solo consideran la matriz de similitud. En una ejecución real también se requieren memoria para el dataset, índices, estructuras de ordenamiento y temporales internos. Además, almacenar toda la matriz es ineficiente porque solo se conservan 10 vecinos por cada vector.

3.4.2. Memoria de la matriz de embeddings

La matriz original X en `float32` ocupa:

$$20\,000 \times 128 \times 4 = 10\,240\,000 \text{ bytes}$$

$$\approx 9,77 \text{ MiB}$$

3.4.3. Memoria de top- k

Para guardar los 10 mejores valores y sus índices por cada vector:

$$20\,000 \times 10 \times 4 = 800\,000 \text{ bytes}$$

para valores, y la misma cantidad para índices enteros de 32 bits. En total:

$$1\,600\,000 \text{ bytes} \approx 1,53 \text{ MiB}$$

3.4.4. Memoria temporal por bloques

En la implementación usada, las filas se particionan entre workers y las columnas se procesan en bloques de tamaño $B = 1024$. Cada worker calcula una submatriz temporal:

$$S_{\text{temp}} = X_{\text{filas worker}} X_{\text{bloque columnas}}^T$$

Con 8 workers, cada worker procesa aproximadamente:

$$\frac{20\,000}{8} = 2\,500$$

filas. La matriz temporal por worker tiene entonces:

$$2\,500 \times 1024 = 2\,560\,000$$

valores `float32`, lo que equivale aproximadamente a 9,77 MiB por worker. La memoria temporal agregada para los 8 workers se mantiene cercana a 78 MiB, sin considerar temporales internos de NumPy. Aun así, este consumo es muy inferior a mantener la matriz completa de similitud.

3.5. Comparación de Estrategias

3.5.1. Estrategia A: matriz completa más ordenamiento

Esta alternativa calcula directamente:

$$S = XX^T$$

Luego ordena cada fila de S para obtener los 10 mayores valores. Su ventaja principal es la simplicidad conceptual y la posibilidad de usar multiplicación matricial altamente optimizada. Sin embargo, presenta tres problemas:

- Materializa 400 000 000 similitudes aunque solo se requieren $20\,000 \times 10$ resultados finales.
- Requiere al menos 1,49 GiB en `float32` solo para la matriz de similitud.
- Ordenar filas completas de 20 000 elementos es innecesario cuando solo se busca top-10.

Por estas razones, se descarta como estrategia final.

3.5.2. Estrategia B: bloques exactos con top- k incremental

Esta alternativa divide el cálculo en bloques. Cada bloque de columnas se compara contra un subconjunto de filas y se actualiza el top-10 de cada vector sin almacenar todas las similitudes. El bloque temporal se descarta luego de actualizar los candidatos.

La estrategia mantiene exactitud porque todos los pares son comparados, pero no materializa la matriz global. Su costo computacional sigue siendo $O(N^2D)$, pero el consumo de memoria se reduce significativamente. Esta es la estrategia seleccionada.

3.5.3. Estrategia C: búsqueda aproximada de vecinos

Una alternativa para escenarios de mayor escala es usar técnicas aproximadas como ANN, HNSW o FAISS. Estas reducen el número de comparaciones y escalan mejor a millones de vectores. Sin embargo, no garantizan recuperar exactamente el top-10 real, por lo que requieren una métrica de calidad como:

$$Recall@10 = \frac{|Top10_{\text{exacto}} \cap Top10_{\text{aproximado}}|}{10}$$

No se selecciona esta alternativa porque el tamaño exigido en el enunciado puede resolverse con una estrategia exacta por bloques.

3.6. Estrategia Seleccionada

La estrategia final consiste en:

1. Generar o cargar una matriz X de $20\,000 \times 128$ en `float32`.
2. Normalizar cada vector a norma 1.
3. Compartir X entre procesos usando `shared_memory`.
4. Particionar las filas entre workers.
5. Recorrer las columnas en bloques de tamaño $B = 1024$.
6. Calcular similitudes parciales con multiplicación matricial:

$$S_{\text{bloque}} = X_{\text{filas}} X_{\text{columnas}}^T$$

7. Eliminar la diagonal asignando $-\infty$ a la comparación x_i contra sí mismo.
8. Actualizar top-10 mediante `argpartition`, sin ordenar todos los candidatos.
9. Retornar los top-10 parciales de cada bloque de filas y construir el resultado global.

3.6.1. Pseudocódigo

Listing 4: Pseudocódigo de la estrategia por bloques.

```

1 Entrada: X normalizada, N, D, K=10, B=1024, workers
2
3 Compartir X en memoria compartida
4
5 Dividir filas de X entre workers
6
7 Para cada worker en paralelo:
8     recibir rango de filas [inicio, fin)
```

```

9      inicializar top_values con -infinito
10     inicializar top_indices con -1
11
12     Para col_start desde 0 hasta N con paso B:
13         col_end = min(col_start + B, N)
14         Xj = X[col_start:col_end]
15
16         sims = X[inicio:fin] @ Xj.T
17
18         Si una fila se compara consigo misma:
19             sims[fila_local, columna_local] = -infinito
20
21         candidatos = concatenar(top_actual, sims)
22         indices = concatenar(indices_actuales, indices_del_bloque)
23
24         conservar solo los K mayores con argpartition
25
26     ordenar top-K final de mayor a menor
27     retornar top_values y top_indices
28
29 Unir resultados parciales de todos los workers

```

3.6.2. Fragmento de Código Relevante

Listing 5: Worker que procesa un rango de filas y actualiza top-k.

```

1  def worker_topk(args):
2      row_start, row_end, N, D, B, K, shm_name = args
3
4      shm = shared_memory.SharedMemory(name=shm_name)
5      X = np.ndarray((N, D), dtype=np.float32, buffer=shm.buf)
6
7      row_count = row_end - row_start
8      Xi = X[row_start:row_end]
9
10     top_values = np.full((row_count, K), -np.inf, dtype=np.float32)
11     top_indices = np.full((row_count, K), -1, dtype=np.int32)
12
13     row_ids = np.arange(row_start, row_end, dtype=np.int32)
14
15     for col_start in range(0, N, B):
16         col_end = min(col_start + B, N)
17         Xj = X[col_start:col_end]
18
19         sims = Xi @ Xj.T
20
21         mask = (row_ids >= col_start) & (row_ids < col_end)
22         if np.any(mask):
23             local_rows = np.where(mask)[0]
24             local_cols = row_ids[mask] - col_start
25             sims[local_rows, local_cols] = -np.inf
26
27         col_indices = np.arange(col_start, col_end, dtype=np.int32)
28         col_indices_matrix = np.broadcast_to(col_indices, sims.shape)
29
30         candidatos_values = np.concatenate((top_values, sims), axis=1)
31         candidatos_indices = np.concatenate((top_indices, col_indices_matrix),
32                                             axis=1)
33
34         pos = np.argpartition(candidatos_values, -K, axis=1)[: , -K:]
35
36         top_values = np.take_along_axis(candidatos_values, pos, axis=1)
37         top_indices = np.take_along_axis(candidatos_indices, pos, axis=1)

```

```

37     orden = np.argsort(-top_values, axis=1)
38     top_values = np.take_along_axis(top_values, orden, axis=1)
39     top_indices = np.take_along_axis(top_indices, orden, axis=1)
40
41     shm.close()
42     return row_start, top_values, top_indices
43

```

3.7. Paralelización, Balance de Carga y Localidad de Memoria

3.7.1. Paralelización

La paralelización se realiza por partición de filas. Cada proceso recibe un rango de filas distinto y calcula el top-10 de esos vectores comparándolos contra todos los bloques de columnas. La matriz X es compartida en modo solo lectura, por lo que no hay condiciones de carrera sobre los embeddings.

Cada worker mantiene sus propios arreglos locales `top_values` y `top_indices`. Como ningún worker escribe el top- k de filas ajenas, no se requiere bloqueo, `critical`, `lock` ni sincronización fina durante el cálculo principal.

3.7.2. Balance de carga

El balance de carga es adecuado porque todos los workers reciben una cantidad similar de filas. Cada fila debe compararse contra los N vectores, por lo que el trabajo asignado a cada worker es aproximadamente:

$$\frac{N}{p} \times N \times D$$

donde p es el número de workers. Para $N = 20\,000$ y $p = 8$, cada worker procesa aproximadamente 2 500 filas contra los 20 000 vectores.

3.7.3. Localidad de memoria

La localidad de memoria mejora porque las columnas se recorren en bloques de 1024 vectores. Cada bloque X_j se reutiliza para calcular similitudes contra todas las filas asignadas al worker. Además, al mantener top- k local se evita escribir una matriz global de $N \times N$ similitudes.

Sin embargo, el algoritmo sigue generando matrices temporales para cada bloque, por lo que el rendimiento puede quedar limitado por ancho de banda de memoria, creación de temporales y operaciones de selección top- k .

3.8. Benchmark y Resultados

3.8.1. Registro del entorno de ejecución

- **Sistema operativo:** Windows, ejecución desde PowerShell.
- **Lenguaje:** Python.
- **Librerías principales:** NumPy, multiprocessing, concurrent.futures.
- **Archivo ejecutado:** Ejercicio_3.py.
- **Tamaño de datos:** $20\,000 \times 128$ embeddings.
- **Tipo de dato:** float32.
- **Top-k:** $K = 10$.
- **Tamaño de bloque:** $B = 1024$.
- **Workers evaluados:** 1, 2, 4 y 8.
- **Repeticiones:** una ejecución por configuración.
- **CPU y RAM:** no registradas explícitamente en la captura; se recomienda agregarlas desde el equipo usado antes de la entrega final.

3.8.2. Métricas utilizadas

Las métricas evaluadas fueron:

$$S_p = \frac{T_1}{T_p}$$

$$E_p = \frac{S_p}{p}$$

$$Throughput = \frac{N(N-1)}{T_p}$$

donde T_1 es el tiempo con un worker, T_p es el tiempo con p workers y $N(N-1)$ corresponde al número de comparaciones excluyendo la diagonal.

3.8.3. Tabla de benchmark

Workers	Tiempo (s)	Speedup	Eficiencia	Throughput comp/s
1	4.1124	1.0000	1.0000	97 263 029.82
2	2.3791	1.7286	0.8643	168 124 738.32
4	1.8001	2.2845	0.5711	222 200 717.06
8	1.6417	2.5049	0.3131	243 636 060.18

Cuadro 8: Resultados experimentales del cálculo de similitud por bloques.

3.8.4. Análisis de resultados

Los resultados muestran una mejora efectiva al aumentar el número de workers. Con 1 worker el tiempo total fue de 4,1124 segundos, mientras que con 8 workers se redujo a 1,6417 segundos. Esto corresponde a un speedup máximo de:

$$S_8 = \frac{4,1124}{1,6417} = 2,5049$$

El throughput también mejora desde 97,26 millones de comparaciones por segundo con 1 worker hasta 243,64 millones de comparaciones por segundo con 8 workers.

Sin embargo, la escalabilidad no es lineal. La eficiencia cae desde 0,8643 con 2 workers hasta 0,3131 con 8 workers. Esto indica que el algoritmo escala, pero comienza a saturarse por overhead de paralelización y presión sobre la memoria. Entre las causas probables están:

- creación de procesos;
- acceso concurrente a memoria compartida;
- matrices temporales `sims`;
- operaciones `concatenate`;
- selección parcial con `argpartition`;
- copias de resultados parciales desde los workers al proceso principal;
- limitaciones del ancho de banda de memoria.

La mejor configuración en tiempo total fue 8 workers. No obstante, desde el punto de vista de eficiencia paralela, 2 workers aprovechan mejor los recursos, ya que entregan $1,7286\times$ de speedup con una eficiencia de 86,43 %.

3.9. Métrica de Calidad

La estrategia implementada es exacta, no aproximada. Todos los vectores son comparados contra todos los demás vectores, y el top-10 se obtiene mediante selección parcial. Por lo tanto, no se requiere una métrica de calidad como `Recall@10`, que sería necesaria si se usara una técnica aproximada.

Aun así, se propone validar la corrección comparando el resultado por bloques contra una implementación de referencia con matriz completa para un subconjunto pequeño, por ejemplo $N = 1000$. Si los índices top-10 coinciden para cada fila, se valida que la estrategia por bloques conserva el resultado exacto sin almacenar toda la matriz.

3.10. Riesgos, Limitaciones y Condiciones de Falla

1. **Costo cuadrático:** aunque se reduce el consumo de memoria, el algoritmo sigue realizando $O(N^2D)$ operaciones. Para datasets de cientos de miles o millones de embeddings, el enfoque exacto puede dejar de ser viable.
2. **Escalabilidad sublineal:** los resultados muestran que aumentar workers no genera speedup proporcional. A partir de cierto punto, el overhead y el ancho de banda de memoria dominan.
3. **Memoria temporal:** aunque no se almacena la matriz completa, cada worker crea matrices temporales. Con bloques demasiado grandes, la memoria puede crecer y afectar el rendimiento.
4. **Tamaño de bloque:** bloques pequeños aumentan overhead de iteración; bloques grandes aumentan uso de memoria. El valor $B = 1024$ representa un compromiso práctico.
5. **Diagonal:** si no se elimina la comparación x_i contra x_i , cada vector aparecería como su propio vecino más similar.
6. **Dependencia del hardware:** el rendimiento depende del número real de núcleos, ancho de banda de memoria, implementación BLAS de NumPy y carga del sistema.
7. **Aproximaciones futuras:** para volúmenes mayores se debería considerar FAISS, HNSW o ANN, midiendo calidad mediante `Recall@10` contra una referencia exacta.

3.11. Conclusiones

1. La matriz completa de similitud requiere aproximadamente 1,49 GiB en `float32` y 2,98 GiB en `float64`, sin contar temporales ni estructuras adicionales.
2. La estrategia por bloques evita materializar la matriz completa y mantiene solo el top-10 por vector.
3. La partición por filas evita condiciones de carrera porque cada worker actualiza únicamente sus propios resultados.
4. El mejor tiempo observado fue 1,6417 segundos con 8 workers, obteniendo un speedup de $2,5049\times$.
5. La eficiencia cae al aumentar workers, lo que evidencia saturación por overhead y memoria. Esto es esperable en un problema con alto volumen de datos y selección top- k incremental.
6. La estrategia seleccionada es exacta; no requiere una métrica de calidad aproximada, aunque se recomienda validar contra una matriz completa en un subconjunto pequeño.

3.12. Declaración de Uso de Herramientas

Se utilizó apoyo de IA generativa para estructurar el informe, revisar la justificación técnica y adaptar la explicación a los criterios de la evaluación. El código fue ejecutado y validado experimentalmente por el grupo, y los resultados incluidos corresponden a la corrida mostrada para el archivo `Ejercicio_3.py`.

3.13. Referencias

1. Grama, A., Gupta, A., Karypis, G., & Kumar, V. (2003). *Introduction to Parallel Computing*. Addison-Wesley.
2. Pacheco, P. (2011). *An Introduction to Parallel Programming*. Morgan Kaufmann.
3. NumPy Developers. *NumPy Documentation*. Disponible en: <https://numpy.org/doc/>
4. Python Software Foundation. *multiprocessing documentation*. Disponible en: <https://docs.python.org/3/library/multiprocessing.html>
5. INF8090 – Computación Paralela y Distribuida. Enunciado de Evaluación 1, Segunda Parte, Ejercicio 3.